



ADITYA UNIVERSITY

SCHOOL OF COMPUTING

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Cornerstone project work entitled “**E-commerce Management System**” is submitted by

Y. Satya sree (25B11CSA51)

K . Lavanya (25B11CS438)

K.Sanjana (25B11CSA79)

K Sindhu (25B11CS408)

in partial fulfillment of the requirements for the award of the B.Tech. degree in Computer Science and Engineering for the academic year 2026-2027.

Project Guide

Mrs. K Vydehi, M.Tech.,(Ph.D),
Assistant Professor,
Department of CSE

Head of the Department

Dr. Phani Sridhar Addepalli, Ph.D,
Assistant Professor,
Department of CSE

DECLARATION

We hereby declare that the Cornerstone project entitled **“E-commerce Management System”** is original and is submitted to **ADITYA UNIVERSITY**, Surampalem, in partial fulfillment of the requirements for the award of the **B.Tech. Degree in Computer Science and Engineering**. We further declare that this project work has not been submitted, either in full or in part, for the award of any degree in this or any other institution. We also confirm that this work complies with the Institutional Academic Integrity and AI Usage Policy

Place: Surampalem

Date: 03-10-2026

By

Y.Satya sree	(25B11CSA51)
K . Lavanya	(25B11CS438)
K.Sanjana	(25B11CSA79)
K Sindhu	(25B11CS408)

ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our Cornerstone Project Guide, **Mrs. K Vydehi, Assistant Professor**, who has guided us throughout the project. Her valuable support, encouragement, and guidance have greatly contributed to the successful completion of this work.

We would like to thank our Deputy Heads of the Department, **Dr. Chandra Sekhar Kolli, Associate Professor, Dr Hari Jyothula, Associate Professor and Dr Pennada Siva Satya Prasad, Assistant Profeesor**, Department of Computer Science and Engineering, for their support and valuable suggestions.

Our deepest thanks to our Head of the Department **Dr. Phani Sridhar Addepalli, Assistant Professor**, for his inspiration and for providing all the necessary facilities and resources required for this project.

We also extend our sincere thanks to **Dr. M.V. Rajesh, Associate Dean, School of Computing** for their encouragement and support during the course of our project.

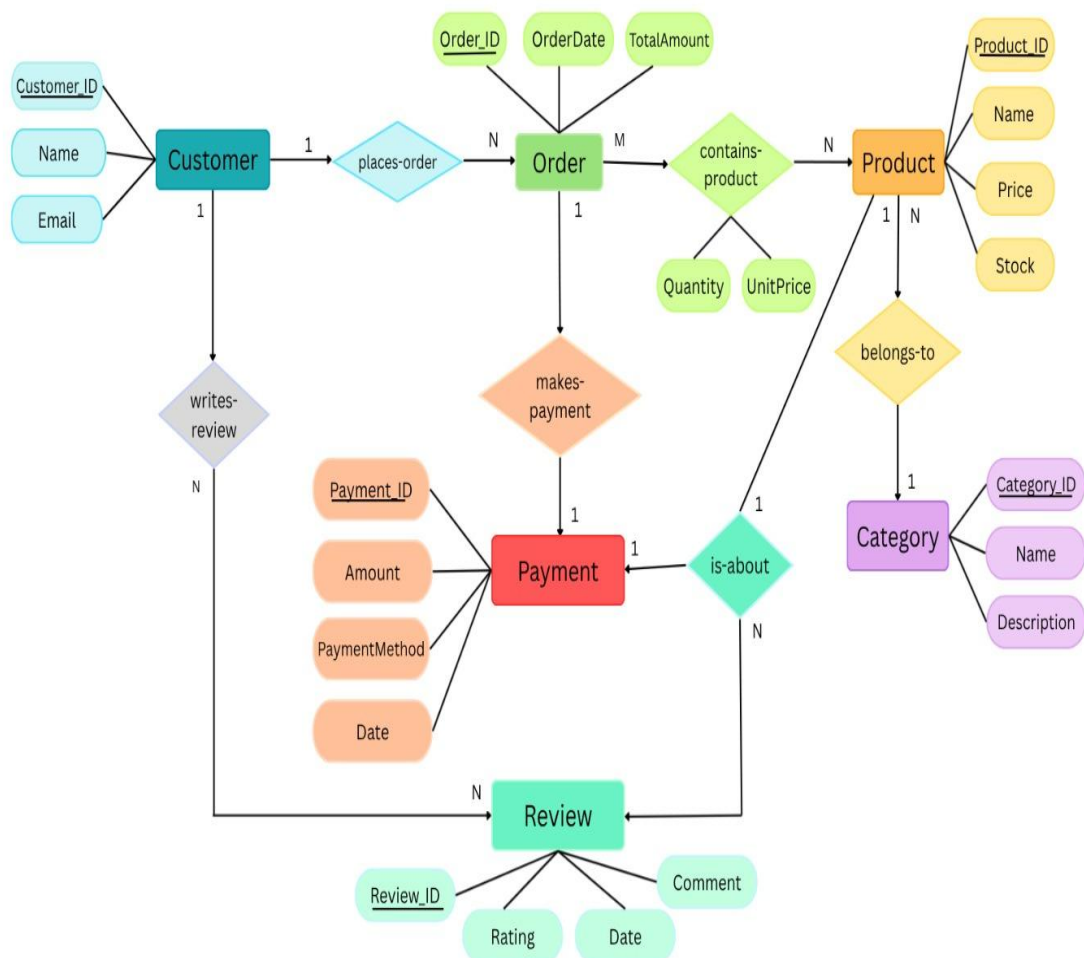
We would like to express our sincere gratitude to **Dr. G. Suresh, Registrar, Dr. A Ramesh, Pro Vice-Chancellor (Engineering & Sciences), Dr. S. Rama Sree, Pro Vice-Chancellor (Academics), Dr. M.B. Srinivas, Vice-Chancellor, Dr. M. Sreenivasa Reddy, Deputy Pro Chancellor and Management, Aditya University** for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank the **faculty members, lab technicians, non-teaching staff, and our friends** who have directly or indirectly supported us in completing this Cornerstone project successfully.

TABLE OF CONTENTS

S.No	Topic	Page Number
1	Abstract	7
2	Introduction	9
3	Database Design	13
4	ER Diagram	12
5	Relational Schema	13
6	Table Design	14–15
7	Normalization	16–20
8	SQL Implementation	21–24
9	SQL Queries and Outputs	25–32
10	Conclusion	33
11	Future Enhancements	34
12	References	35

LIST OF FIGURES



LIST OF TABLES

Table 1.	Major entities and their purposes
Table 2.	Relationships and cardinalities
Table 3.	Relational schema
Table 4.	CUSTOMER table structure
Table 5.	ORDER table structure
Table 6.	PRODUCT table structure
Table 7.	CATEGORY table structure
Table 8.	PAYMENT table structure
Table 9.	REVIEW table structure
Table 10.	ORDER_ITEM table structure
Table 11.	Sample SQL query outputs



ACRONYMS

DBMS – Database Management System

ER – Entity Relationship

SQL – Structured Query Language

DDL – Data Definition Language

DML – Data Manipulation Language

DQL – Data Query Language

DCL – Data Control Language

TCL – Transaction Control Language

PK – Primary Key

FK – Foreign Key

M:N – Many-to-Many

1:N – One-to-Many

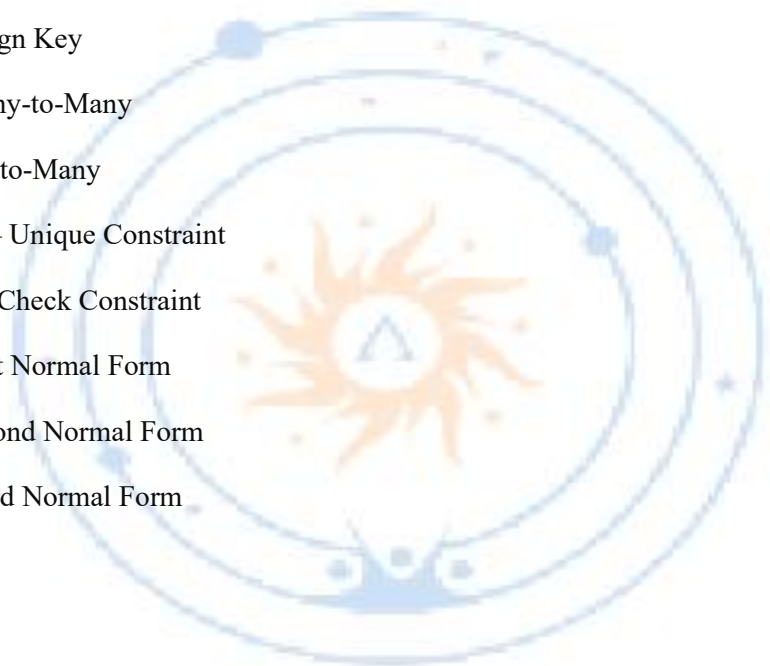
UNIQUE – Unique Constraint

CHECK – Check Constraint

1NF – First Normal Form

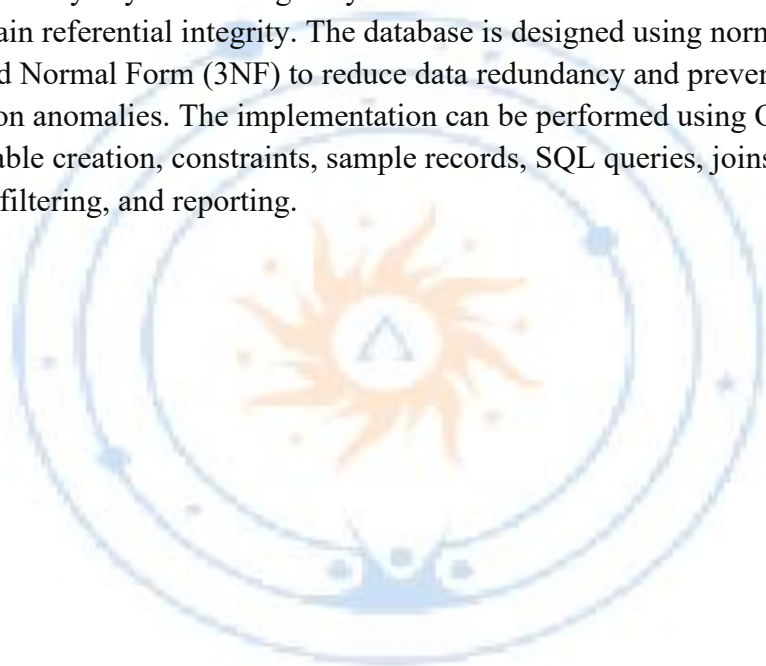
2NF – Second Normal Form

3NF – Third Normal Form



1. ABSTRACT

The E-Commerce Order Management System is a centralized relational database designed to efficiently manage customers, orders, products, product categories, payments, and customer reviews. The system maintains customer information and records the orders placed by customers. Each order can contain multiple products, and each product can occur in multiple orders. Therefore, the many-to-many relationship between Order and Product is represented using an intermediate ORDER_ITEM table that stores details such as Quantity and UnitPrice. The database also manages product categories, order payments, and customer reviews. Primary keys and foreign keys are used to establish relationships between tables and maintain referential integrity. The database is designed using normalization principles up to Third Normal Form (3NF) to reduce data redundancy and prevent insertion, update, and deletion anomalies. The implementation can be performed using Oracle SQL and includes table creation, constraints, sample records, SQL queries, joins, aggregate functions, grouping, filtering, and reporting.



2. INTRODUCTION

2.1 Project Overview

The E-Commerce Order Management System provides a structured database for managing online shopping operations.

The system stores customer details, product information, categories, orders, payments, and reviews. It allows an organization to maintain all important e-commerce information in a centralized database.

The system represents the major activities of an online shopping platform:

Customer → Places Order → Order contains Products → Product belongs to Category
and

Order → Makes Payment
and

Customer → Writes Review → Product

2.2 Problem Statement

In a manual or poorly organized e-commerce system:

- Customer information may be stored in multiple places.
- Order details can be difficult to track.
- Product stock information may become inconsistent.
- Payment records may not be properly connected with orders.
- Product categories may be duplicated.
- Customer reviews may not be properly linked to products.
- Repeated order and product information can cause redundancy.
- Updating product information in multiple records can create inconsistencies.

The proposed database solves these problems by storing information in related and normalized tables.

2.3 Objectives

- Maintain customer information in a centralized database.
- Store and manage product details.
- Organize products according to categories.
- Record orders placed by customers.
- Store multiple products in an order.
- Maintain quantity and unit price for each ordered product.
- Track payments associated with orders.
- Allow customers to provide product reviews.
- Maintain primary key and foreign key relationships.
- Reduce data redundancy through normalization.
- Design the database up to Third Normal Form (3NF).
- Generate useful reports using SQL queries.

2.4 Scope

The scope of the E-Commerce Order Management System includes:

- Customer management
- Product management
- Category management
- Order management
- Order item management
- Payment management
- Review management
- Product stock management
- Customer order tracking
- Payment tracking
- Product review tracking
- SQL-based reporting

3.1 Existing System

The existing/manual system may involve:

- Manual customer records.
- Separate product and order records.
- Repeated product information.
- Difficult order tracking.
- Manual payment tracking.
- Difficulty maintaining product stock.
- Unorganized customer reviews.
- Data inconsistency due to duplicate information.

3.2 Proposed System

The proposed system uses a centralized relational database.

The database contains separate tables for:

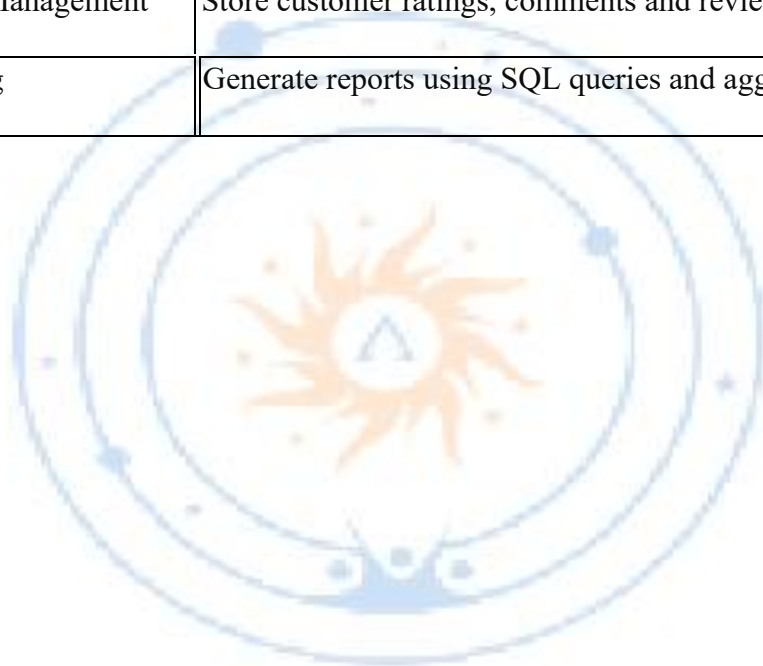
- CUSTOMER
- ORDER
- PRODUCT
- CATEGORY
- PAYMENT
- REVIEW
- ORDER_ITEM

The tables are connected using primary and foreign keys.

The ORDER_ITEM table resolves the many-to-many relationship between orders and products.

3.3 Functional Requirements

	Functional Requirement
Customer Management	Add, update, search and display customer details.
Product Management	Store product name, price and stock information.
Category Management	Maintain product categories and descriptions.
Order Management	Create and maintain customer orders.
Order Item Management	Store products, quantity and unit price for each order.
Payment Management	Record payment amount, method and date.
Review Management	Store customer ratings, comments and review dates.
Reporting	Generate reports using SQL queries and aggregate functions.



3.4 Non-Functional Requirements

- Data Integrity: Primary keys and foreign keys maintain correct relationships.
- Security: Database access should be restricted to authorized users.
- Reliability: Order and payment information should be stored consistently.
- Performance: Queries should efficiently retrieve customer, order and product information.
- Scalability: The database should support increasing numbers of customers, products and orders.
- Usability: The system should be easy for administrators to manage.

3.5 End Users

User	Main Interaction
Customer	Browse products, place orders and write reviews.
E-Commerce Administrator	Manage customers, products, categories and orders.
Order Manager	Track orders and order items.
Payment Administrator	Manage payment records.
Product Manager	Manage product price, stock and category information.
System Administrator	Manage database users, security and backups.

4. DATABASE DESIGN

Entity	Purpose
Customer	Stores customer information.
Order	Stores orders placed by customers.
Product	Stores product details, price and stock.
Category	Stores product category information.
Payment	Stores payment details for orders.
Review	Stores customer ratings and comments about products.
Order_Item	Stores products included in each order along with quantity and unit price.

4. DATABASE DESIGN

4.1 Major Entities

Entity	Purpose
Customer	Stores customer information.
Order	Stores orders placed by customers.
Product	Stores product details, price and stock.
Category	Stores product category information.
Payment	Stores payment details for orders.
Review	Stores customer ratings and comments about products.
Order_Item	Stores products included in each order along with quantity and unit price.

4.2 Relationships and Cardinality

Relationship	Cardinality	Description
Customer — Order	1:N	One customer can place many orders.
Order — Product	M:N	One order can contain many products and one product can appear in many orders.
Product — Category	N:1	Many products can belong to one category.
Order — Payment	1:1	Each order makes a payment in the supplied ER model.
Customer — Review	1:N	One customer can write multiple reviews.
Product — Review	1:N	One product can receive multiple reviews.

Order–Product Relationship

The ER model shows:

Order M : N Product

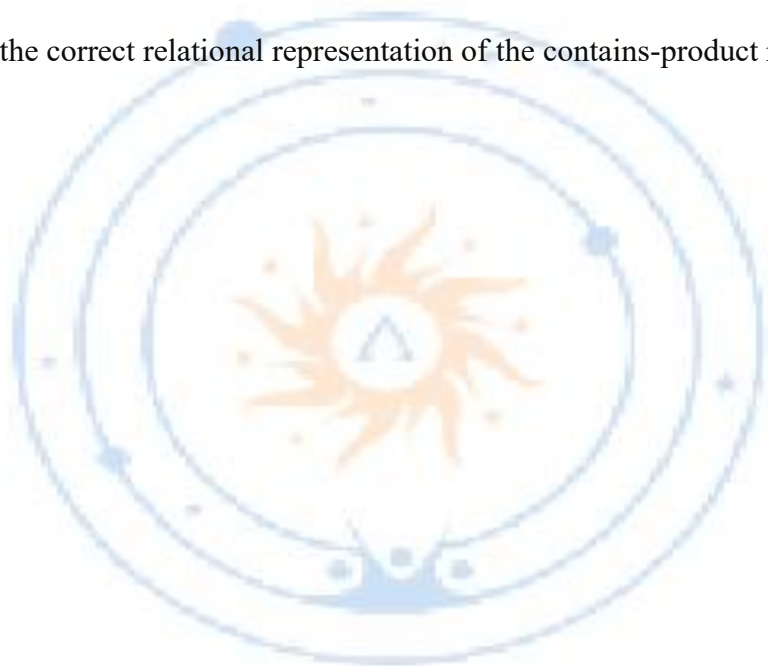
with relationship attributes:

- Quantity
- UnitPrice

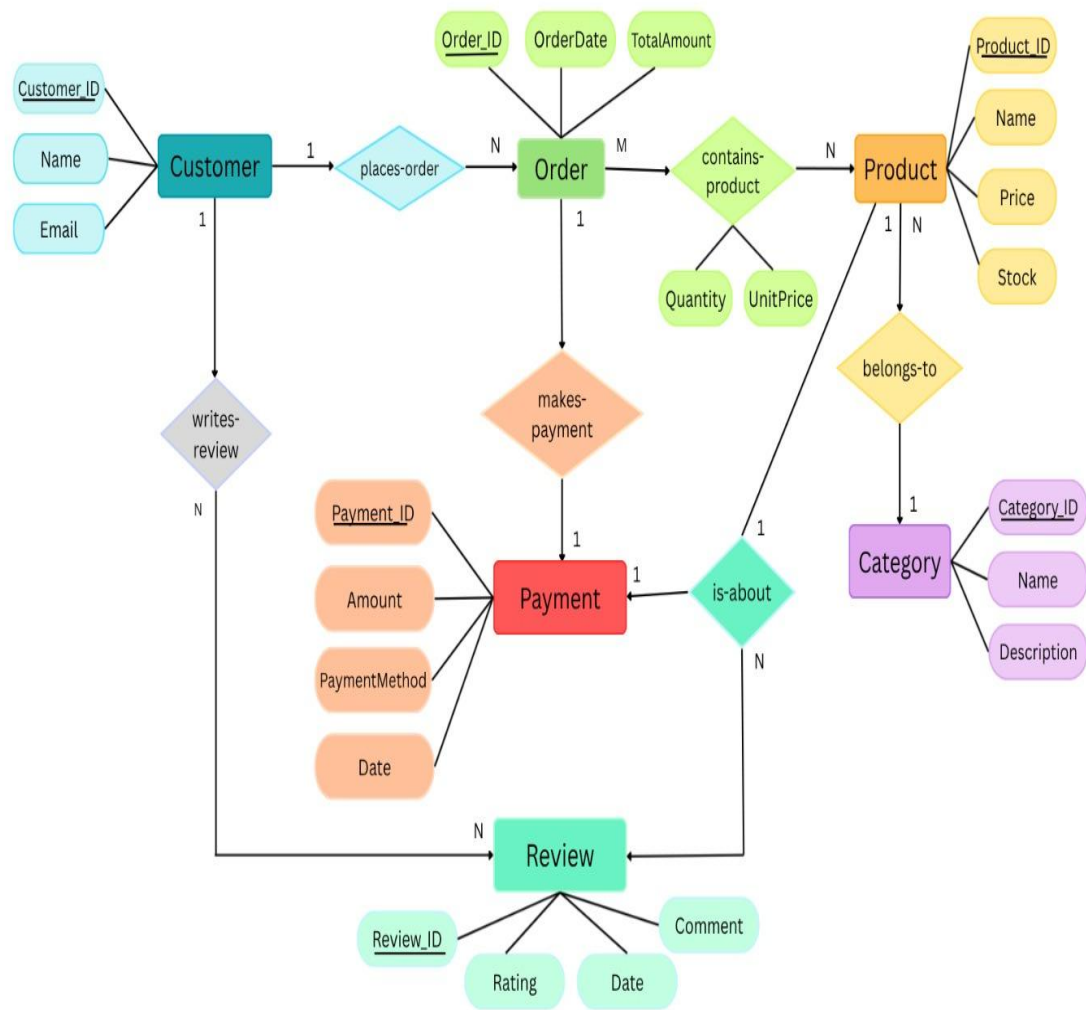
Therefore, the relationship is converted into:

ORDER_ITEM(Order_ID, Product_ID, Quantity, UnitPrice)

This is the correct relational representation of the contains-product relationship.



5. ER DIAGRAM



6. RELATIONAL SCHEMA

Relation	Relational Schema
CUSTOMER	Customer_ID (PK), Name, Email
ORDER	Order_ID (PK), OrderDate, TotalAmount, Customer_ID (FK)
PRODUCT	Product_ID (PK), Name, Price, Stock, Category_ID (FK)
CATEGORY	Category_ID (PK), Name, Description
PAYMENT	Payment_ID (PK), Amount, PaymentMethod, Date, Order_ID (FK)
REVIEW	Review_ID (PK), Rating, Comment, Date, Customer_ID (FK), Product_ID (FK)
ORDER_ITEM	Order_ID (PK/FK), Product_ID (PK/FK), Quantity, UnitPrice

7. TABLE DESIGN

7.1 CUSTOMER

Attribute	Data Type	Constraint	Description
Customer_ID	NUMBER	PK	Unique customer identifier
Name	VARCHAR2(50)	NOT NULL	Customer name
Email	VARCHAR2(100)	UNIQUE	Customer email

7.2 ORDER

Attribute	Data Type	Constraint	Description
Order_ID	NUMBER	PK	Unique order identifier
OrderDate	DATE	NOT NULL	Date of order
TotalAmount	NUMBER(10,2)	CHECK	Total order amount
Customer_ID	NUMBER	FK	Customer who placed the order

7.3 PRODUCT

Attribute	Data Type	Constraint	Description
Product_ID	NUMBER	PK	Unique product identifier
Name	VARCHAR2(50)	NOT NULL	Product name
Price	NUMBER(10,2)	CHECK	Product price
Stock	NUMBER	CHECK	Available stock
Category_ID	NUMBER	FK	Product category

7.4 CATEGORY

Attribute	Data Type	Constraint	Description
Category_ID	NUMBER	PK	Unique category identifier
Name	VARCHAR2(50)	NOT NULL	Category name
Description	VARCHAR2(200)	—	Category description

7.5 PAYMENT

Attribute	Data Type	Constraint	Description
Payment_ID	NUMBER	PK	Unique payment identifier
Amount	NUMBER(10,2)	CHECK	Payment amount
PaymentMethod	VARCHAR2(30)	NOT NULL	Payment method
Date	DATE	NOT NULL	Payment date
Order_ID	NUMBER	FK	Related order

7.6 REVIEW

Attribute	Data Type	Constraint	Description
Review_ID	NUMBER	PK	Unique review identifier
Rating	NUMBER	CHECK	Customer rating
Comment	VARCHAR2(200)	—	Customer comment
Date	DATE	NOT NULL	Review date
Customer_ID	NUMBER	FK	Customer who writes review
Product_ID	NUMBER	FK	Product being reviewed

7.7 ORDER_ITEM

Attribute	Data Type	Constraint	Description
Order_ID	NUMBER	PK, FK	Related order
Product_ID	NUMBER	PK, FK	Related product
Quantity	NUMBER	CHECK	Quantity ordered
UnitPrice	NUMBER(10,2)	CHECK	Price of product at order time



8. NORMALIZATION

1NF – First Normal Form

1NF means each column should contain only one value.

There should be no repeated or multiple values in a single cell.

Example: In our E-Commerce system, each order should have separate product details.

2NF – Second Normal Form

2NF means the table should be in 1NF and all data should depend on the complete primary key.

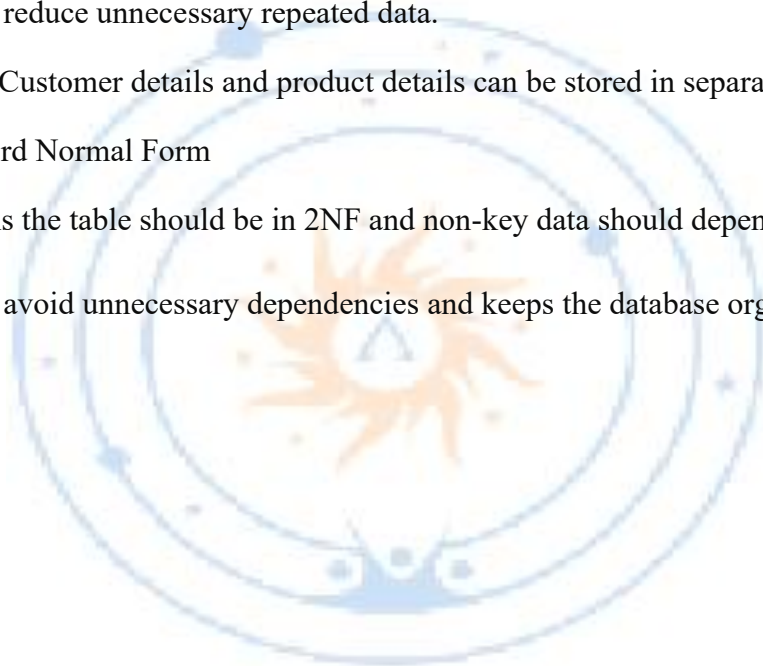
It helps to reduce unnecessary repeated data.

Example: Customer details and product details can be stored in separate tables.

3NF – Third Normal Form

3NF means the table should be in 2NF and non-key data should depend only on the primary key.

It helps to avoid unnecessary dependencies and keeps the database organized.



8.1 1NF – First Normal Form

ORDER_ID	CUSTOMER_ID	CUSTOMER_NAME	PRODUCT_ID	QUANTITY
O101	C01	Ravi	P101	2
O102	C02	Priya	P102	1
O103	C03	Anu	P103	3
O104	C01	Ravi	P104	1
O105	C04	Kiran	P101	2
O106	C05	Sita	P105	1

ORDER_ID → CUSTOMER_ID, CUSTOMER_NAME

PRODUCT_ID → PRODUCT_NAME, PRICE

(ORDER_ID, PRODUCT_ID) → QUANTITY

PRIMARY KEY: (ORDER_ID, PRODUCT_ID)

Redundancy :In 1NF, all values are atomic, but some customer details may be repeated when the same customer places multiple orders. This can cause data redundancy.

8.2 Second Normal Form (2NF)

Orders Table

ORDER_ID	CUSTOMER_ID	CUSTOMER_NAME
O101	C01	Ravi
O102	C02	Priya
O103	C03	Anu
O104	C01	Ravi
O105	C04	Kiran
O106	C05	Sita

Order_Details Table

ORDER_ID	PRODUCT_ID	QUANTITY
O101	P101	2
O102	P102	1
O103	P103	3
O104	P104	1
O105	P101	2
O106	P105	1

In the 1NF table, customer details and order details are stored together. Here, CUSTOMER_NAME depends only on CUSTOMER_ID, not on the complete key. This creates partial dependency and data repetition.

Redundancy: 2nf partial dependency is removed by separating customer details from order details. Therefore, repeated customer information is reduced and redundancy is minimized.

In our E-Commerce Order Management System, the data is decomposed into two tables: CUSTOMER and ORDER_DETAILS. The CUSTOMER table stores customer information, while the ORDER_DETAILS table stores order and product information. In the original table, CUSTOMER_NAME depends only on CUSTOMER_ID, which causes partial dependency and data redundancy. To remove this dependency, CUSTOMER_NAME is moved to the CUSTOMER table. The ORDER_DETAILS table contains ORDER_ID, CUSTOMER_ID, PRODUCT_ID, and QUANTITY. Thus, partial dependency is removed, data redundancy is reduced, and the tables are converted into Second Normal Form (2NF).

8.3 Third Normal Form (3NF)

Customer Table

CUSTOMER_ID	CUSTOMER_NAME
C01	Ravi
C02	Priya
C03	Anu
C04	Kiran
C05	Sita

Orders Table

ORDER_ID	PRODUCT_ID	QUANTITY
O101	P101	2
O102	P102	1
O103	P103	3
O104	P104	1
O105	P101	2
O106	P105	1

3NF is used to remove transitive dependency from the tables.

In our E-Commerce Order Management System, we separate Customer, Product, and Order information.

Customer_ID is the primary key in the Customer table, so customer details depend only on Customer_ID.

Product_ID is the primary key in the Product table, so product details depend only on Product_ID.

Order_ID is the primary key in the Order table, and Customer_ID connects the order with the customer.

This separation avoids data duplication and update problem

9. SQL IMPLEMENTATION

9.1 DDL — Table Creation

CUSTOMER

```
CREATE TABLE Customer
(
    Customer_ID NUMBER PRIMARY KEY,
    Name VARCHAR2(50) NOT NULL,
    Email VARCHAR2(100) UNIQUE
);
```

CATEGORY

```
CREATE TABLE Category
(
    Category_ID NUMBER PRIMARY KEY,
    Name VARCHAR2(50) NOT NULL,
    Description VARCHAR2(200)
);
```

PRODUCT

```
CREATE TABLE Product
(
    Product_ID NUMBER PRIMARY KEY,
    Name VARCHAR2(50) NOT NULL,
    Price NUMBER(10,2) CHECK (Price >= 0),
    Stock NUMBER CHECK (Stock >= 0),
    Category_ID NUMBER,
    CONSTRAINT product_category_fk
    FOREIGN KEY (Category_ID)
    REFERENCES Category(Category_ID)
);
```

);

ORDER

CREATE TABLE Orders

(

Order_ID NUMBER PRIMARY KEY,

OrderDate DATE NOT NULL,

TotalAmount NUMBER(10,2) CHECK (TotalAmount >= 0),

Customer_ID NUMBER,

CONSTRAINT order_customer_fk

FOREIGN KEY (Customer_ID)

REFERENCES Customer(Customer_ID)

);

Note: Orders is used instead of Order because ORDER can conflict with SQL syntax.

PAYMENT

CREATE TABLE Payment

(

Payment_ID NUMBER PRIMARY KEY,

Amount NUMBER(10,2) CHECK (Amount >= 0),

PaymentMethod VARCHAR2(30) NOT NULL,

Date DATE NOT NULL,

Order_ID NUMBER,

CONSTRAINT payment_order_fk

FOREIGN KEY (Order_ID)

REFERENCES Orders(Order_ID)

);

REVIEW

CREATE TABLE Review

(

Review_ID NUMBER PRIMARY KEY,

Rating NUMBER CHECK (Rating BETWEEN 1 AND 5),

Comment VARCHAR2(200),

Date DATE NOT NULL,

Customer_ID NUMBER,

Product_ID NUMBER,

CONSTRAINT review_customer_fk

FOREIGN KEY (Customer_ID)

REFERENCES Customer(Customer_ID),

CONSTRAINT review_product_fk

FOREIGN KEY (Product_ID)

REFERENCES Product(Product_ID)

);

ORDER_ITEM

CREATE TABLE Order_Item

(

Order_ID NUMBER,

Product_ID NUMBER,

Quantity NUMBER CHECK (Quantity > 0),

UnitPrice NUMBER(10,2) CHECK (UnitPrice >= 0),

CONSTRAINT order_item_pk

PRIMARY KEY (Order_ID, Product_ID),

CONSTRAINT order_item_order_fk

FOREIGN KEY (Order_ID)

REFERENCES Orders(Order_ID),

CONSTRAINT order_item_product_fk

FOREIGN KEY (Product_ID)

REFERENCES Product(Product_ID)

);

9.2 Sample Data

Customer

```
INSERT INTO Customer
```

```
VALUES (1, 'Ravi Kumar', 'ravi@gmail.com');
```

```
INSERT INTO Customer
```

```
VALUES (2, 'Priya Sharma', 'priya@gmail.com');
```

Category

```
INSERT INTO Category
```

```
VALUES (101, 'Electronics', 'Electronic products');
```

```
INSERT INTO Category
```

```
VALUES (102, 'Clothing', 'Fashion and clothing products');
```

Product

```
INSERT INTO Product
```

```
VALUES (1001, 'Laptop', 55000, 20, 101);
```

```
INSERT INTO Product
```

```
VALUES (1002, 'Headphones', 2500, 50, 101);
```

Orders

```
INSERT INTO Orders
```

```
VALUES (5001, DATE '2026-09-01', 57500, 1);
```

```
INSERT INTO Orders
```

```
VALUES (5002, DATE '2026-09-02', 2500, 2);
```

Order_Item

```
INSERT INTO Order_Item  
VALUES (5001, 1001, 1, 55000);
```

```
INSERT INTO Order_Item  
VALUES (5001, 1002, 1, 2500);
```

Payment

```
INSERT INTO Payment  
VALUES (9001, 57500, 'UPI', DATE '2026-09-01', 5001);
```

Review

```
INSERT INTO Review VALUES(7001, 5, 'Excellent product', DATE '2026-09-05', 1,  
1001);
```



10. SQL QUERIES AND OUTPUTS

Q1. Display all customers

```
SELECT *  
  
FROM Customer;
```

Q2. Display all products

```
SELECT *  
  
FROM Product;
```

Q3. Display products with price greater than 1000

```
SELECT Product_ID, Name, Price  
  
FROM Product  
  
WHERE Price > 1000;
```

Purpose: Uses the WHERE clause for filtering.

Q4. Display customers and their orders

```
SELECT  
  
    c.Customer_ID,  
    c.Name,  
    o.Order_ID,  
    o.OrderDate,  
    o.TotalAmount  
  
FROM Customer c  
  
JOIN Orders o  
  
ON c.Customer_ID = o.Customer_ID;
```

Q5. Display products with their categories

```
SELECT  
  
    p.Product_ID,  
    p.Name AS Product_Name,  
    p.Price,  
  
    c.Name AS Category_Name
```

```
FROM Product p
JOIN Category c
ON p.Category_ID = c.Category_ID;
```

Q6. Display orders above ₹10,000

```
SELECT
    Order_ID,
    OrderDate,
    TotalAmount
FROM Orders
WHERE TotalAmount > 10000;
```

Q7. Display products having stock greater than 10

```
SELECT
    Product_ID,
    Name,
    Stock
FROM Product
WHERE Stock > 10;
```

ADVANCED QUERIES

Q8. Display order details with products

```
SELECT
    o.Order_ID,
    o.OrderDate,
    p.Name AS Product_Name,
    oi.Quantity,
    oi.UnitPrice
FROM Orders o
JOIN Order_Item oi
ON o.Order_ID = oi.Order_ID
```


JOIN Product p

ON oi.Product_ID = p.Product_ID;

Q9. Calculate total quantity sold for each product

SELECT

p.Product_ID,

p.Name,

SUM(oi.Quantity) AS Total_Quantity

FROM Product p

JOIN Order_Item oi

ON p.Product_ID = oi.Product_ID

GROUP BY p.Product_ID, p.Name;

Q10. Find total sales amount

SELECT

SUM(TotalAmount) AS Total_Sales

FROM Orders.

Q11. Find average order amount

SELECT

AVG(TotalAmount) AS Average_Order_Amount

FROM Orders;

Q12. Count orders placed by each customer

SELECT

c.Customer_ID,

c.Name,

COUNT(o.Order_ID) AS Total_Orders

FROM Customer c

LEFT JOIN Orders o

ON c.Customer_ID = o.Customer_ID

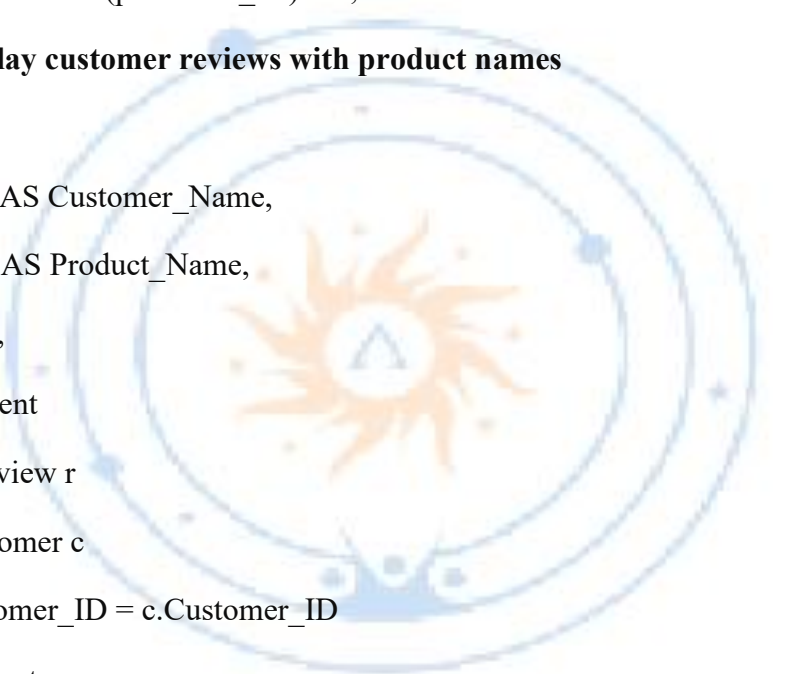
GROUP BY c.Customer_ID, c.Name;

Q13. Find categories having more than 2 products

```
SELECT  
  
    c.Category_ID,  
  
    c.Name,  
  
    COUNT(p.Product_ID) AS Product_Count  
  
FROM Category c  
  
JOIN Product p  
  
ON c.Category_ID = p.Category_ID  
  
GROUP BY c.Category_ID, c.Name  
  
HAVING COUNT(p.Product_ID) > 2;
```

Q14. Display customer reviews with product names

```
SELECT  
  
    c.Name AS Customer_Name,  
  
    p.Name AS Product_Name,  
  
    r.Rating,  
  
    r.Comment  
  
FROM Review r  
  
JOIN Customer c  
  
ON r.Customer_ID = c.Customer_ID  
  
JOIN Product p  
  
ON r.Product_ID = p.Product_ID;
```



Q15. Find the highest-rated products

SELECT

p.Product_ID,

p.Name,

AVG(r.Rating) AS Average_Rating

FROM Product p

JOIN Review r

ON p.Product_ID = r.Product_ID

GROUP BY p.Product_ID, p.Name

HAVING AVG(r.Rating) >= 4;



10.1 Sample SQL Query Outputs

Query / Measure	Example Result
Total Customers	Number of customer records
Total Products	Number of product records
Total Categories	Number of category records
Total Orders	Number of orders
Total Sales	Sum of order amounts
Average Order Amount	Average of TotalAmount
Products with Stock > 10	Products satisfying stock condition
Orders Above ₹10,000	Orders satisfying amount condition
Total Quantity Sold	Sum of ordered quantities
Customer Reviews	Review records with customer and product names

11. CONCLUSION

The E-Commerce Order Management System provides a structured relational database for managing customers, products, categories, orders, payments and reviews.

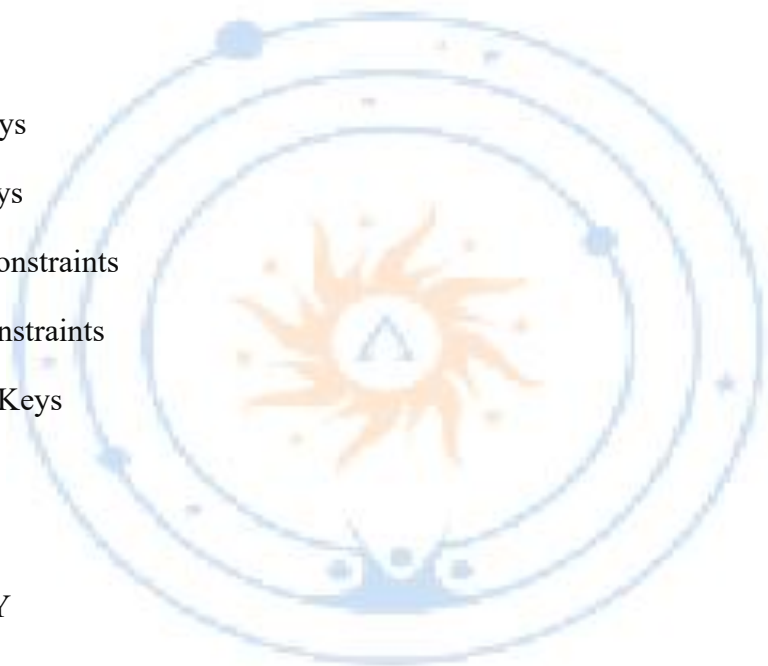
The ER model represents the important relationships between these entities. The many-to-many relationship between Order and Product is resolved using the ORDER_ITEM table, which stores Quantity and UnitPrice.

The database uses primary keys and foreign keys to maintain referential integrity. The design is normalized up to Third Normal Form (3NF), which reduces redundancy and prevents insertion, update and deletion anomalies.

The SQL implementation demonstrates important DBMS concepts including:

- DDL
- DML
- Primary Keys
- Foreign Keys
- UNIQUE constraints
- CHECK constraints
- Composite Keys
- JOIN
- WHERE
- GROUP BY
- HAVING
- COUNT
- SUM
- AVG

The system can therefore provide useful information about customers, orders, products, payments and reviews.



12. FUTURE ENHANCEMENTS

The E-Commerce Order Management System can be enhanced with:

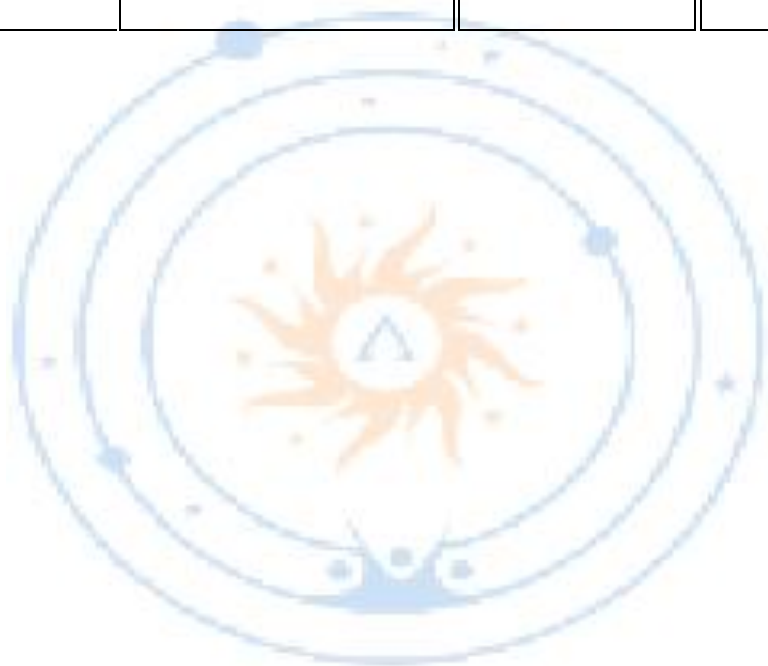
- Shopping cart management.
- Customer login and authentication.
- Online payment gateway integration.
- Order delivery tracking.
- Order cancellation and returns.
- Refund management.
- Discount and coupon management.
- Product wishlist.
- Multiple product images.
- Product search and filtering.
- Inventory management.
- Automatic stock updates after orders.
- Email/SMS order notifications.
- Admin dashboard.
- Customer recommendation system.
- Sales analytics and reports.

13. REFERENCES

1. Silberschatz, Korth and Sudarshan, Database System Concepts, McGraw Hill.
2. Elmasri and Navathe, Fundamentals of Database Systems, Pearson.
3. Oracle Database SQL documentation.
4. Course notes and laboratory manual for Database Management Systems.
5. E-Commerce Order Management System ER Model supplied for the project.

APPENDIX A — RELATIONSHIP SUMMARY

Entity 1	Relationship	Entity 2	Cardinality
Customer	places-order	Order	1:N
Order	contains-product	Product	M:N
Product	belongs-to	Category	N:1
Order	makes-payment	Payment	1:1
Customer	writes-review	Review	1:N
Product	is-about	Review	1:N



APPENDIX B — project team

K . Lakshimi sindhu priya – 25B11CS408

K.Lavanya – 25B11CS438

Y. Satya sree – 25B11CSA51

K.Sanjana – 25B11CSA79

